

Allocazione della memoria

Come e quando viene allocato lo spazio in memoria per gli elementi del programma

ha effetto sugli algoritmi di risoluzione dei riferimenti non locali

allocazione statica

memoria allocata a tempo di compilazione

FORTRAN ✓

COBOL

dobbiamo sapere a priori
quanta memoria serve
NO RICORSIONE

allocazione dinamica

memoria allocata a tempo di esecuzione

- stack (LIFO) ←

- heap

Stack: Allocazione dinamica su Pila

Struttura dati: allocata dinamicamente e gestita in modalità LIFO

=> Per ogni chiamata procedura viene creato un RDA (Recordi di Attivazione)

↳ insieme di informazioni che permettono di eseguire la chiamata procedura e permettono di ritornare correttamente allo stato valido al momento della chiamata

RDA: • indirizzo di ritorno

dimensione nota staticamente

• variabili locali (parametri formali, ambiente dichiarato nella procedura)

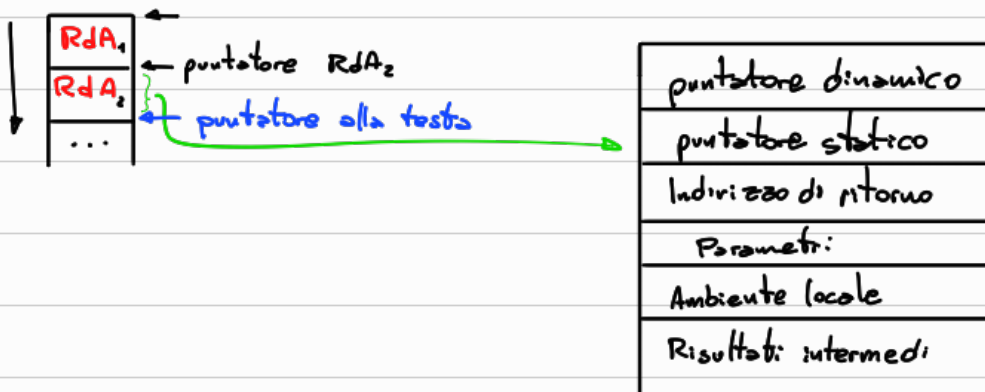
• puntatore alla testa della pila

• altre informazioni specifiche (puntatore statico, puntatore dinamico)

↓
puntatore al RDA della procedura genitore nella catena storica delle definizioni

→ può essere distante nello stack dell'RDA corrente

[0:22:00]



Procedura di chiamata a funzione

- Valutano i parametri (in funzione del metodo di passaggio)
- Allezza RdA sullo stack con ambiente locale
- Salva lo stato del chiamante al momento della chiamata
- Trasferimento del controllo alla procedura chiamata

Procedura al ritorno della chiamata di una funzione

- Recupero informazioni dal RdA (passaggio valore-risultato)
- Deallocazione del RdA
- Recupero dello stato del chiamante
- Ritorno del controllo al chiamante

Implementazione scoping statico

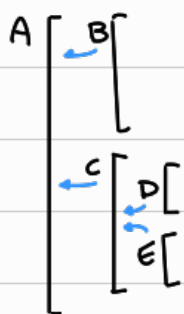
Costruzione del link statico alla chiamata di ogni procedura

Link statico punta all'indirizzo del RdA della procedura che contiene la definizione della procedura in atto.

CS ma catena statica

Profondità statica (Sd) = è il valore che denota la profondità di annidamento

A, B, C, D, E nomi procedure
A → B → C → D → E → C seq. di chiamate
link dinamici
determina link dinamici



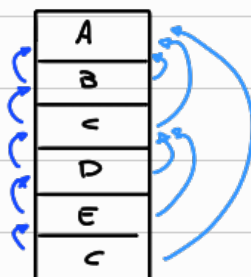
$$Sd(A) = 0$$

$$Sd(B) = 1 + Sd(A) = 1 = Sd(C)$$

$$Sd(D) = 1 + Sd(C) = 2 = Sd(E)$$

B e C sono definiti dentro A

D ed E sono definiti dentro C



Ch $\rightarrow$ denota chiamante

Quali informazioni statiche abbiamo per calcolare il link statico

$\rightarrow$ profondità di annidamento

$\rightarrow$ RDA della procedura (struttura nota staticamente)

Ch chiama P:

[0:54:00]

dobbiamo determinare link statico di P

$\rightarrow$ se P è definito in Ch (link statico = link dinamico)

$\rightarrow$ se P è definito in un blocco che si trova n passi prima di Ch dobbiamo risalire la catena statica di n passi

$$n = Sd(Ch) - Sd(P) + 1$$

• se $n=0$ def P e quindi il link statico per P è l'indirizzo di chiamante

• se $n>0$ $CS(P) = \underbrace{CS}_{n \text{ volte}}(CS \dots CS(Ch))$ il link statico si ottiene risalendo n volte la catena statica (la seq. di link statici)

[1:02:00]

[ESEMPI] [1:19:00]

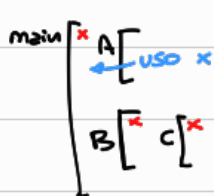
Risoluzione dei riferimenti (non locali)

Note: se il riferimento è locale $\rightarrow$ la risoluzione è ovvia nell'RDA attivo

Se abbiamo x usato in P ma non locale, dobbiamo individuare il blocco più vicino nella catena statica che def x .

$\rightarrow$ Usiamo Sd e l'informazione data dai blocchi che definiscono x

$\rightarrow$ Usiamo la catena statica



Usiamo x in A $\Rightarrow$ cerchiamo nella catena di annidamento, risolvendo da A, il blocco più vicino che def x

il blocco più vicino ad A che def x è il main $\Rightarrow$ lo sappiamo a tempo di compilazione

$$N_x = Sd(A) - Sd(\text{main}) = 1 - 0 = 1$$

$\uparrow$ uso x $\uparrow$ def x (più vicino ad A)

$\downarrow$ quanto dobbiamo risalire per trovare x